

Logical Link Control Sub Layer

Lecture#27-32



Dr. Sanjeev Patel

Asst Professor, CSE Dept.

NIT Rourkela

Outline

- Link Layer Services
- Error Detection
- Parity Check
- Cyclic Redundancy Check
- Hamming Code
- Flow control : Stop and Wait, Sliding Window Protocol
- Error control : ARQ mechanism

Link Layer Services(Design Issues)

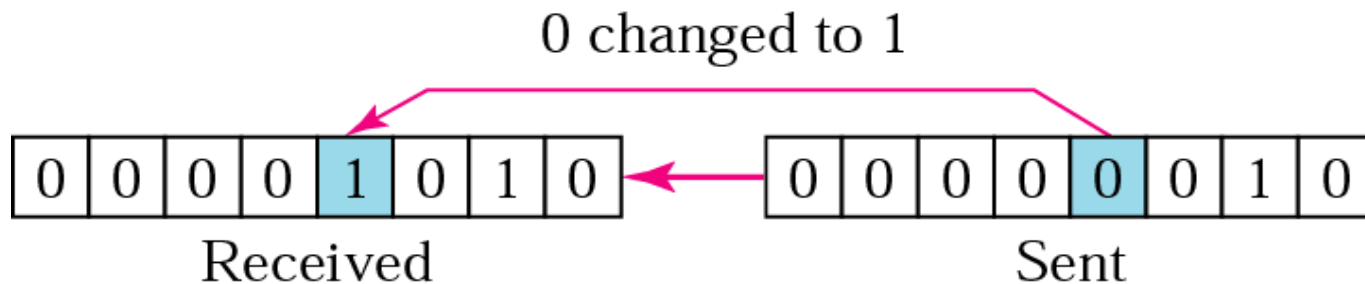
- **Framing, link access:**
 - encapsulate datagram into frame, adding header, trailer
 - channel access if shared medium
- ***Flow Control:***
 - pacing between adjacent sending and receiving nodes
- ***Error Detection:***
 - errors caused by signal attenuation, noise, and receiver detects presence of errors:
- **Error Correction:**
 - receiver identifies *and corrects* bit error(s) without resorting to retransmission
- ***Half-duplex and full-duplex***
 - with half duplex, nodes at both ends of link can transmit, but not at same time

Error Detection

- Error are likely to occur even when there is a sufficient SNR to provide a low bit error rate
- To control error and improve the reliability of a communication line, first we must be able to detect the errors
- CRC one of the best techniques to detect the error
- **Bit Error Rate(BER):** It is the ratio of number N_e of errors appearing over a certain time interval t to the number N_t of 1 and 0 pulses transmitted during this interval. $BER = N_e / N_t = N_e / R * t$

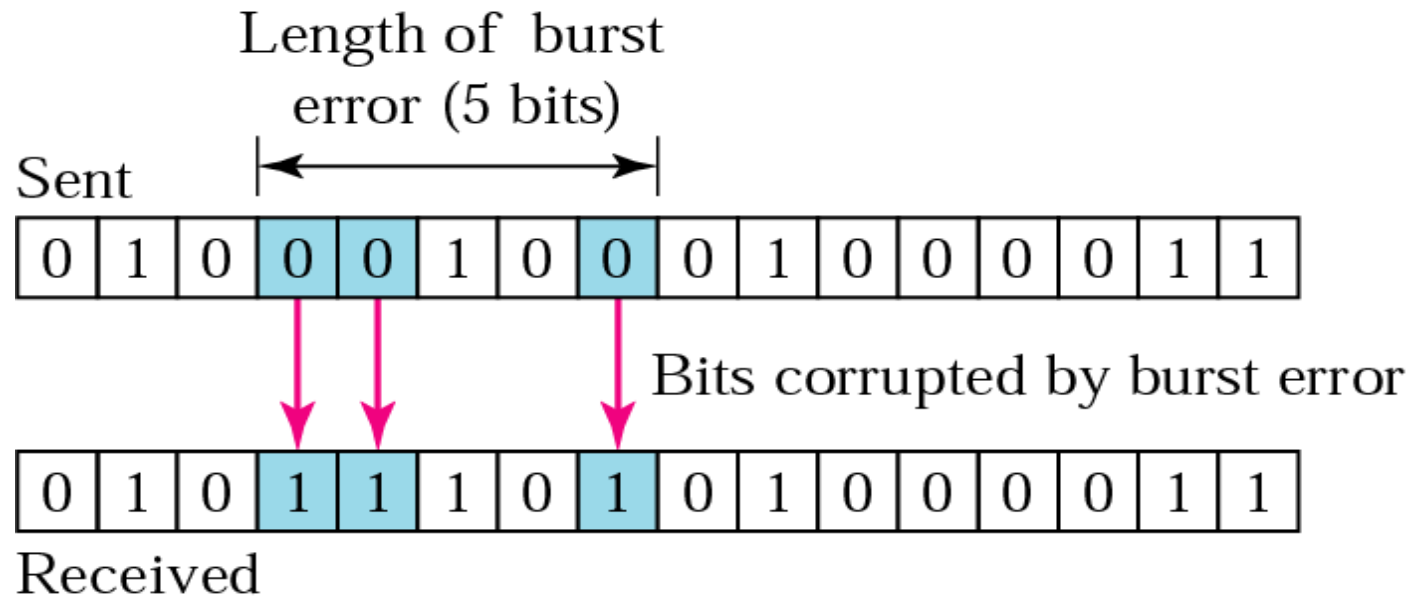
Types of Error

- ***Single Bit Error:*** In a single-bit error, only one bit in the data unit has changed



Burst Error [5]

- A burst error means that 2 or more bits in the data unit have changed*



Detection

Redundancy: *adding extra bits for detecting errors at the destination.*

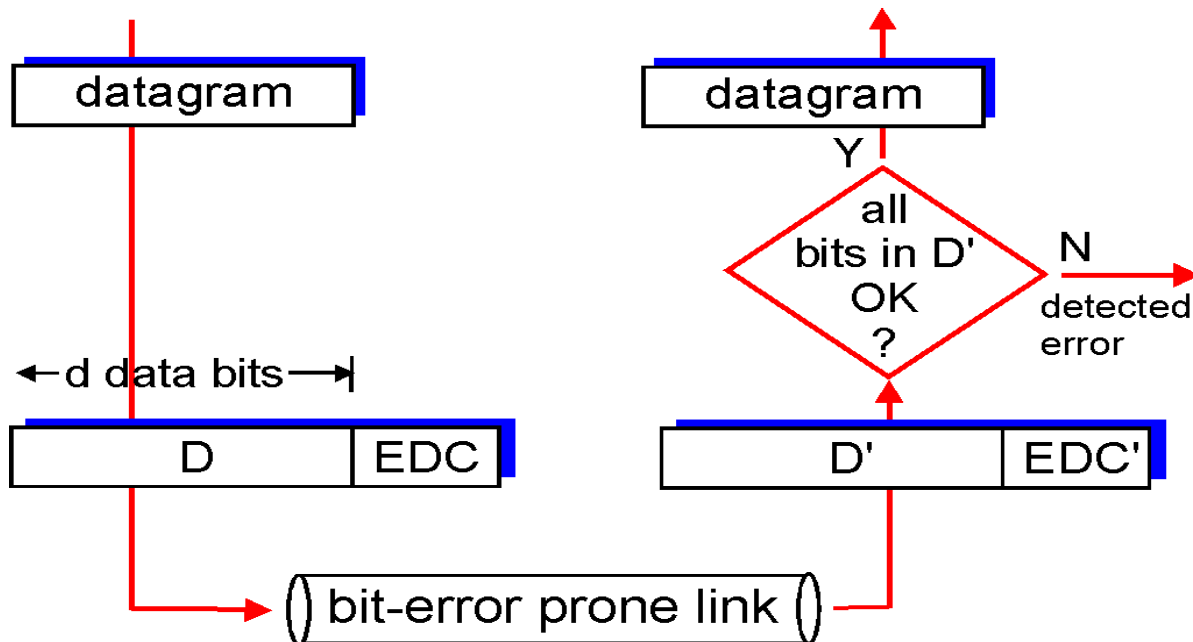
- **Parity Check:** *a parity bit is added to every data unit so that the total number of 1s is even (or odd for odd-parity).*
- **Cyclic Redundancy Check (CRC)**
- **Checksum**

Error Detection [6]

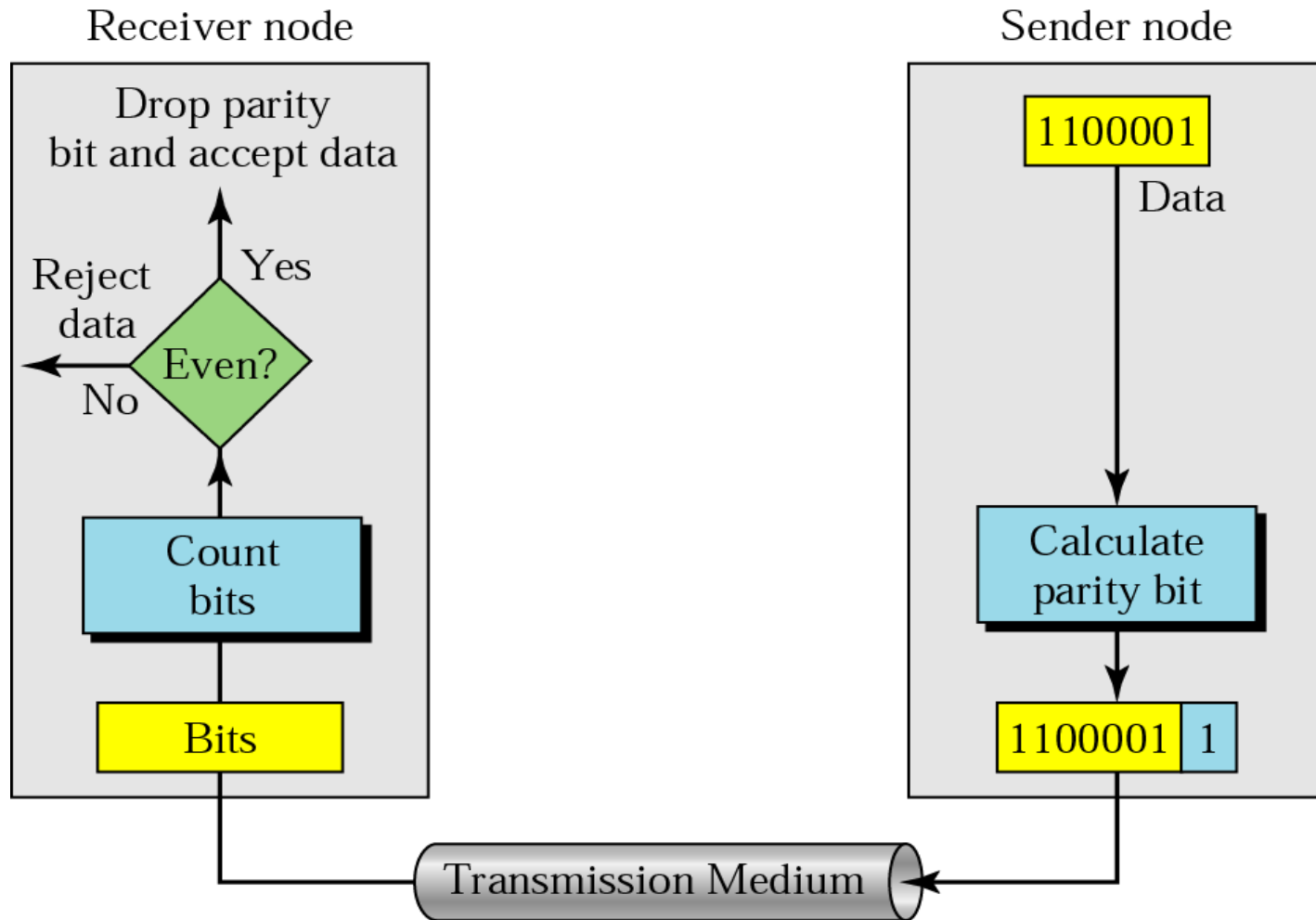
EDC= Error Detection and Correction bits (redundancy)

D = Data protected by error checking, may include header fields

- Error detection not 100% reliable!
 - protocol may miss some errors, but rarely
 - larger EDC field yields better detection and correction



10.5 Even-parity concept [5]



Example 1 [5]

Suppose the sender wants to send the word *world*. In ASCII the five characters are coded as

1110111 1101111 1110010 1101100 1100100

The following shows the actual bits sent

11101110 11011110 11100100 11011000 11001001

Example 2 [5]

Now suppose the word world in Example 1 is received by the receiver without being corrupted in transmission.

11101110 11011110 11100100 11011000 11001001

The receiver counts the 1s in each character and comes up with even numbers (6, 6, 4, 4, 4). The data are accepted.

Example 3 [5]

Now suppose the word world in Example 1 is corrupted during transmission.

11111110 11011110 11101100 11011000 11001001

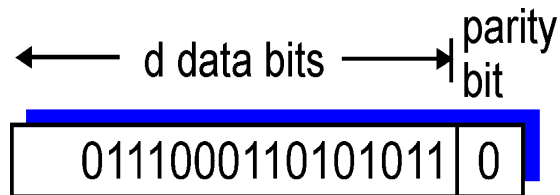
The receiver counts the 1s in each character and comes up with even and odd numbers (7, 6, 5, 4, 4). The receiver knows that the data are corrupted, discards them, and asks for retransmission.

Note: Simple parity check can detect all single-bit errors. It can detect burst errors only if the total number of errors in each data unit is odd

Parity Checking [6]

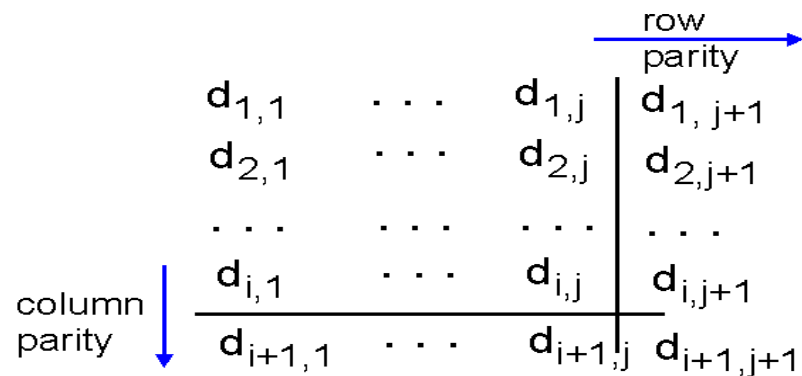
Single Bit Parity:

Detect single bit errors



Two Dimensional Bit Parity:

Detect *and correct* single bit errors



1	0	1	0	1
1	1	1	1	0
0	1	1	1	0
0	0	1	0	1

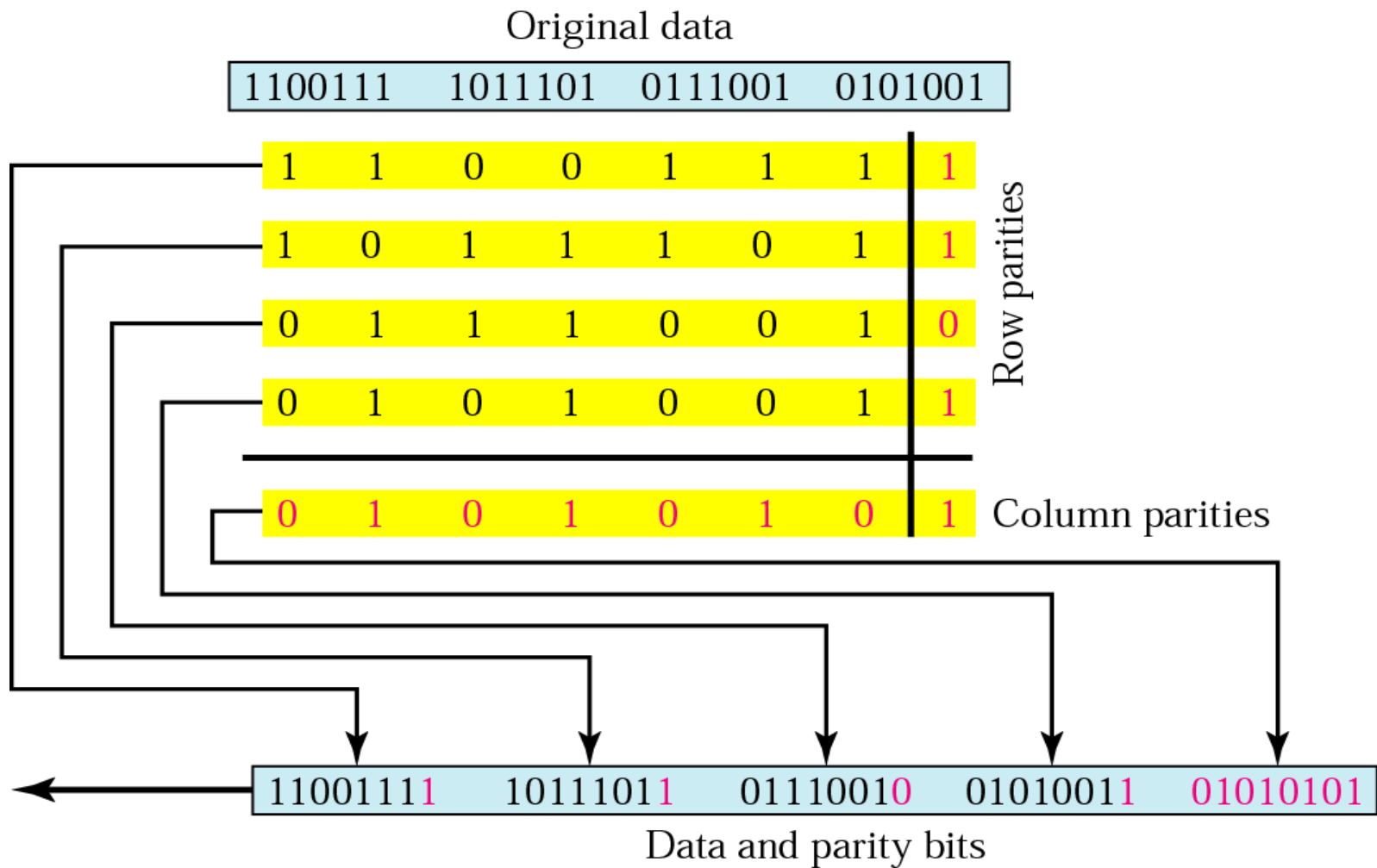
no errors

1	0	1	0	1
1	1	1	1	0
0	1	1	1	0
0	0	1	0	1

parity
error

*correctable
single bit error*

10.6 Two-dimensional parity [5]



Example 4 [5]

Suppose the following block is sent:

10101001 00111001 11011101 11100111 10101010

However, it is hit by a burst noise of length 8, and some bits are corrupted.

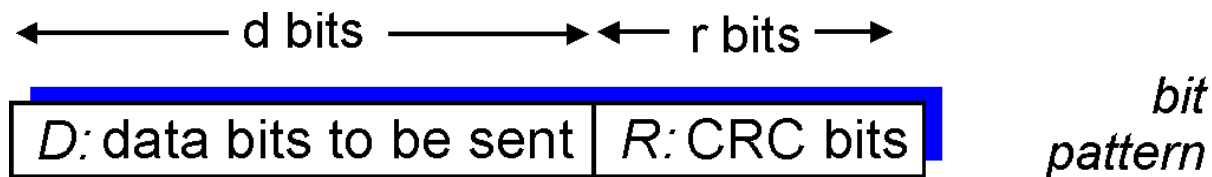
10100011 10001001 11011101 11100111 10101010

When the receiver checks the parity bits, some of the bits do not follow the even-parity rule and the whole block is discarded.

10100011 10001001 11011101 11100111 10101010

Checksumming: Cyclic Redundancy Check [6]

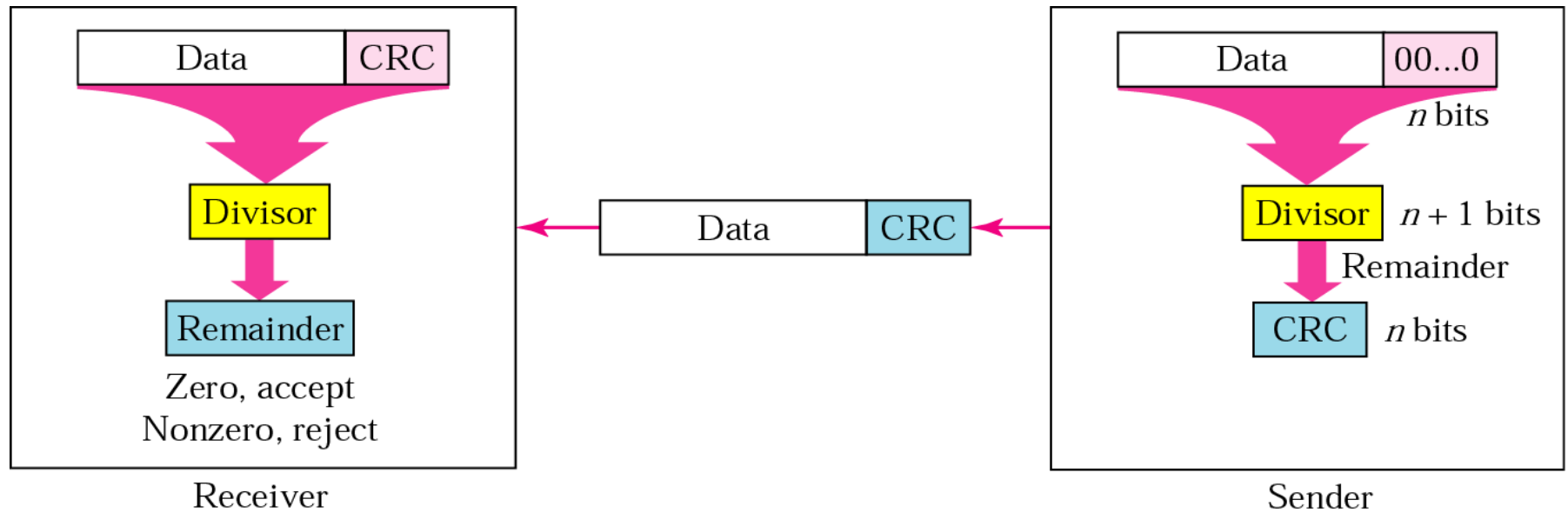
- view data bits, **D**, as a binary number
- choose $r+1$ bit pattern (generator), **G**
- goal: choose r CRC bits, **R**, such that
 - $\langle D, R \rangle$ exactly divisible by G (modulo 2)
 - receiver knows G , divides $\langle D, R \rangle$ by G . If non-zero remainder: error detected!
 - can detect all burst errors less than $r+1$ bits
- widely used in practice (ATM, HDLC)



$$D * 2^r \text{ XOR } R$$

mathematical formula

10.7 CRC generator and checker [5]



CRC Example 5 [6]

Want:

$$D \cdot 2^r \text{ XOR } R = nG$$

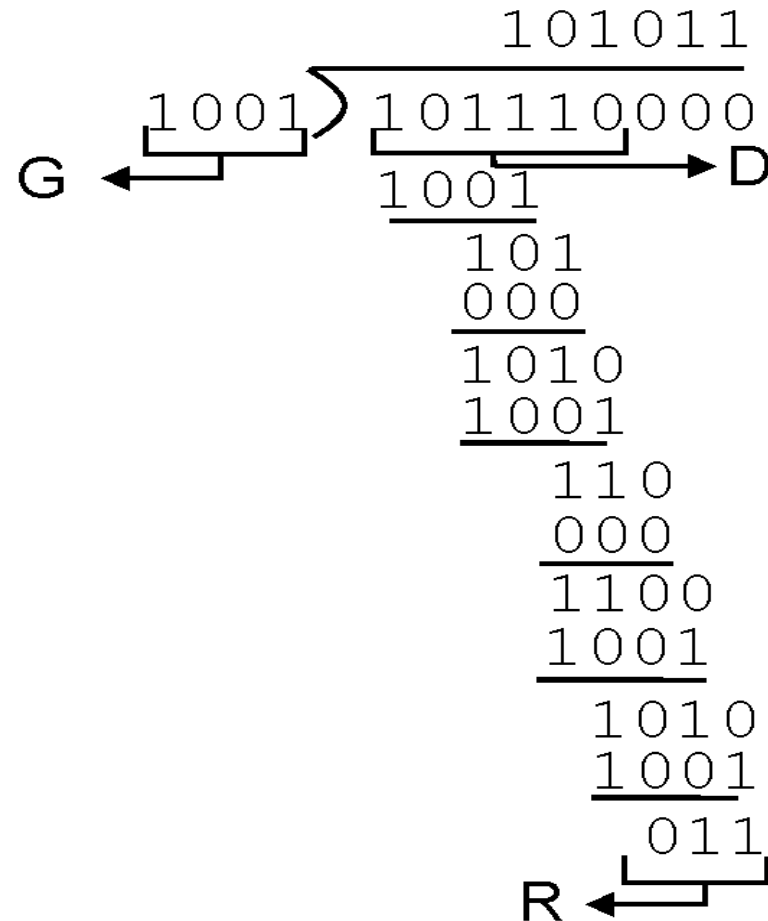
equivalently:

$$D \cdot 2^r = nG \text{ XOR } R$$

equivalently:

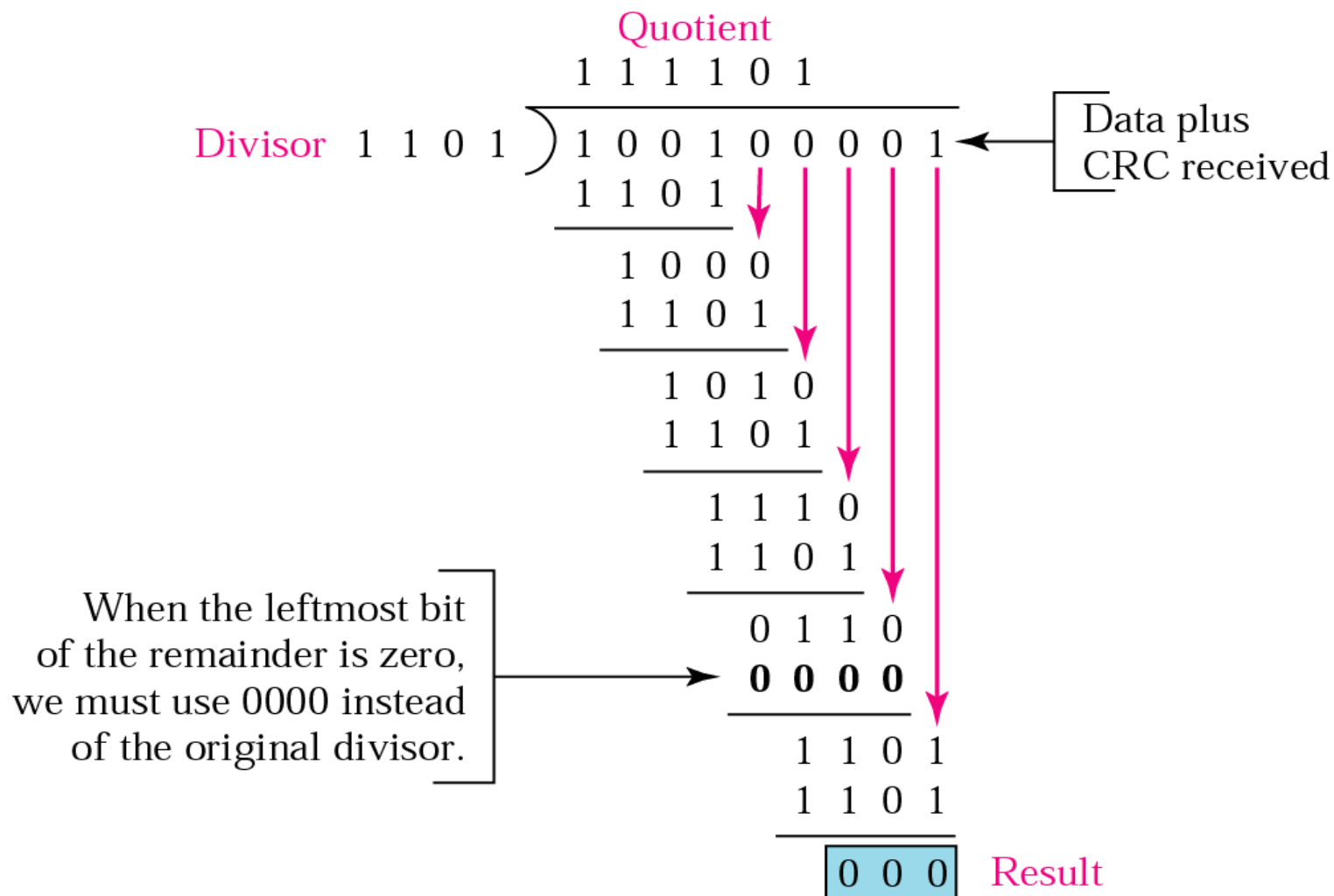
if we divide $D \cdot 2^r$ by G ,
want remainder R

$$R = \text{remainder} \left[\frac{D \cdot 2^r}{G} \right]$$





Example 7 for CRC checker [5]



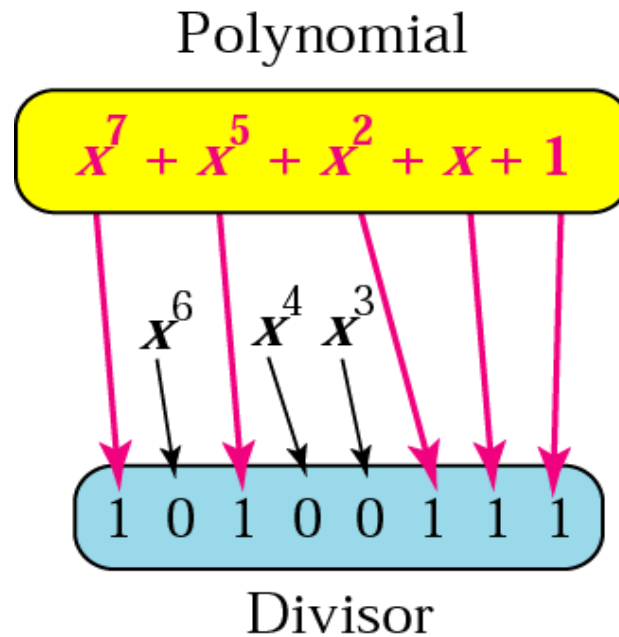


Table 10.1 Standard polynomials [5]

Name	Polynomial	Application
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
ITU-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
ITU-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

Example 8 [5]

Polynomial Properties(Divisor):

1. It should not be divisible by x
2. It should be divisible by $x+1$

It is obvious that we cannot choose x (binary 10) or $x^2 + x$ (binary 110) as the polynomial because both are divisible by x . However, we can choose $x + 1$ (binary 11) because it is not divisible by x , but is divisible by $x + 1$. We can also choose $x^2 + 1$ (binary 101) because it is divisible by $x + 1$ (binary division).

Example 9 [5]

The CRC-12

$$x^{12} + x^{11} + x^3 + x + 1$$

which has a degree of 12, will detect all burst errors affecting an odd number of bits, will detect all burst errors with a length less than or equal to 12, and will detect, 99.97 percent of the time, burst errors with a length of 12 or more.

Error Correction

Retransmission

Forward Error Correction(FEC)

Burst Error Correction

Hamming Code

- Hamming Code-
 - It is error detection and correction technique used at data link layer
 - It is developed by Richard W. Hamming in 1950
- Hamming Distance-
 - It is measure of altered bit or error among the binary numbers and determines the number of bits which is not matched
 - Hamming distance between two words (of the same size) is the number of differences between the corresponding bits.
 - Hamming distance between 111 and 000 is 3
i.e. $(d(111, 000) = 3)$

Data and redundancy bits [5]

Number of data bits m	Number of redundancy bits r	Total bits m + r
1	2	3
2	3	5
3	3	6
4	3	7
5	4	9
6	4	10
7	4	11

Find min $r : (2^r > m+r)$

FEC [5]

Positions of redundancy bits in Hamming code

Each r bit is the parity bit for one combination of data bits

11	10	9	8	7	6	5	4	3	2	1
d	d	d	r_8	d	d	d	r_4	d	r_2	r_1

r_1 : 1, 3, 5, 7, 9, 11, ...

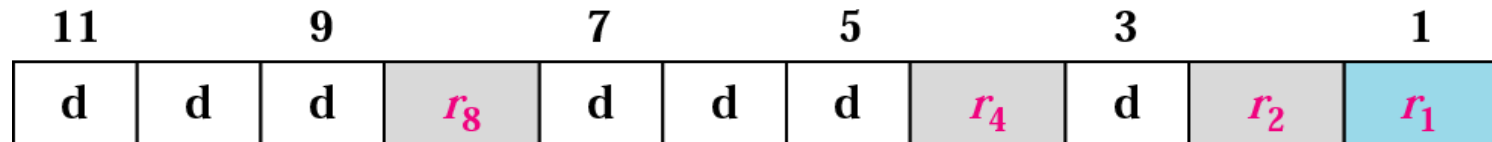
r_2 : 2, 3, 6, 7, 10, 11, ...

r_4 : 4, 5, 6, 7, ...

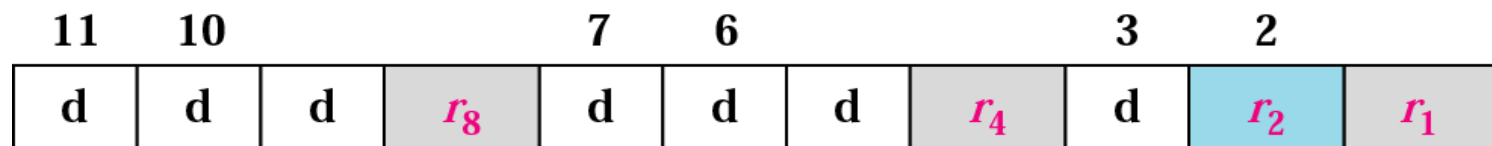
r_8 : 8, 9, 10, 11, ...

Redundancy bits calculation [5]

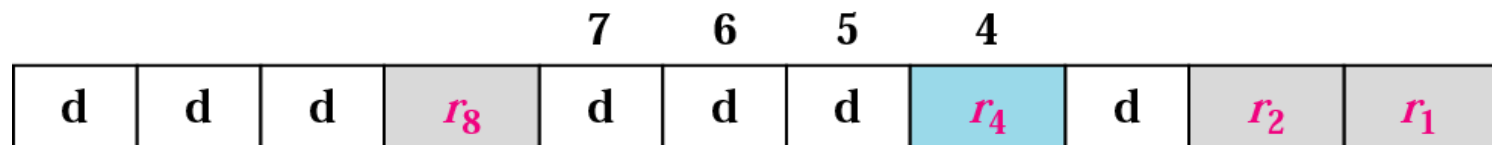
r_1 will take care of these bits.



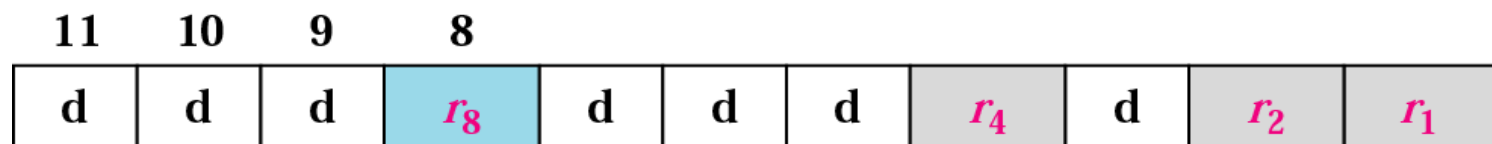
r_2 will take care of these bits.



r_4 will take care of these bits.



r_8 will take care of these bits.



Example of redundancy bit calculation [5]

Data:
1 0 0 1 1 0 1

1	0	0		1	1	0		1		
11	10	9	8	7	6	5	4	3	2	1

Adding r_1

1	0	0		1	1	0		1		1
11	10	9	8	7	6	5	4	3	2	1

Adding r_2

1	0	0		1	1	0		1	0	1
11	10	9	8	7	6	5	4	3	2	1

Adding r_4

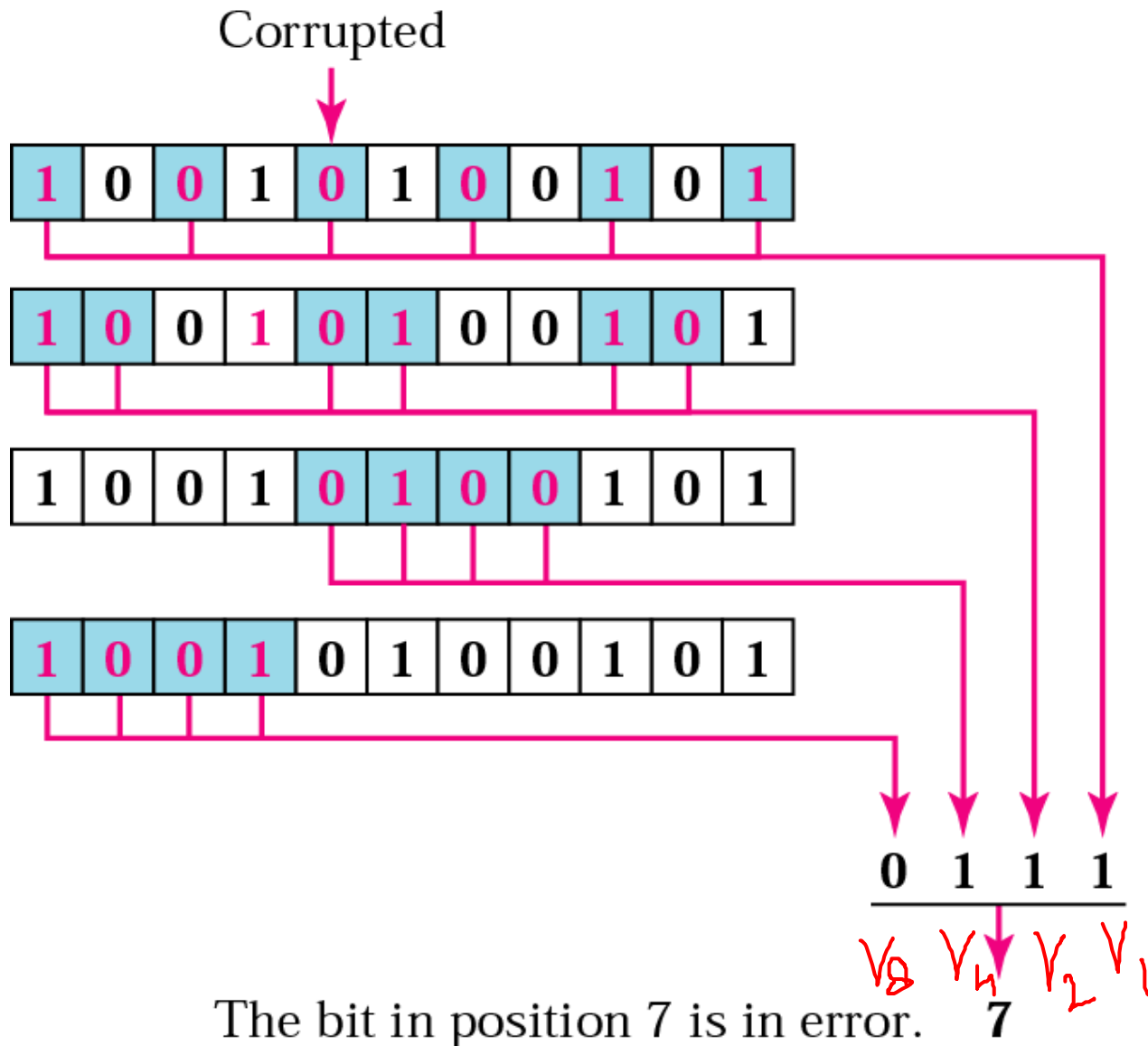
1	0	0		1	1	0	0	1	0	1
11	10	9	8	7	6	5	4	3	2	1

Adding r_8

1	0	0	1	1	1	0	0	1	0	1
11	10	9	8	7	6	5	4	3	2	1

Code:
1 0 0 1 1 1 0 0 1 0 1

Error detection using Hamming code [5]



10.18 Burst error correction example

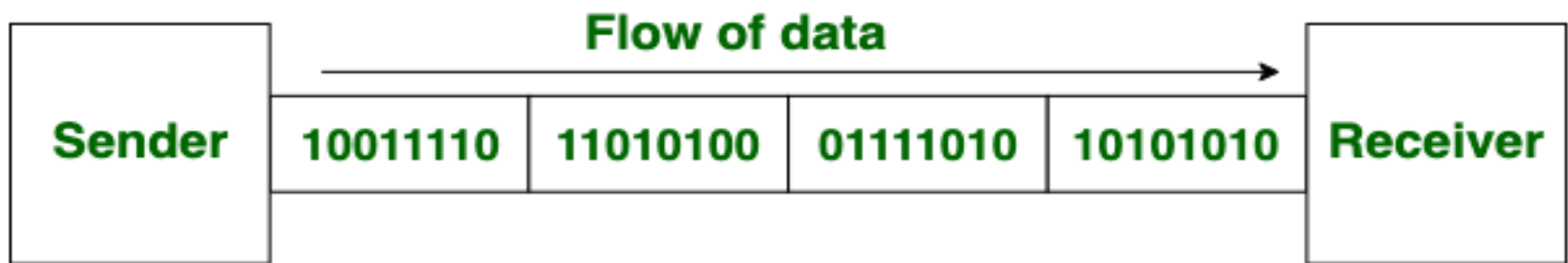
Error → 1111?000011
 Error → 1010?011111
 11111001100
 Error → 011?1011001
 Error → 011?1010110
 Error → 011?1001111
 Received data



11111000011
 10101011111
 11111001100
 01101011001
 01101010110
 01111001111
 Data before being sent

Synchronous Transmission [9]

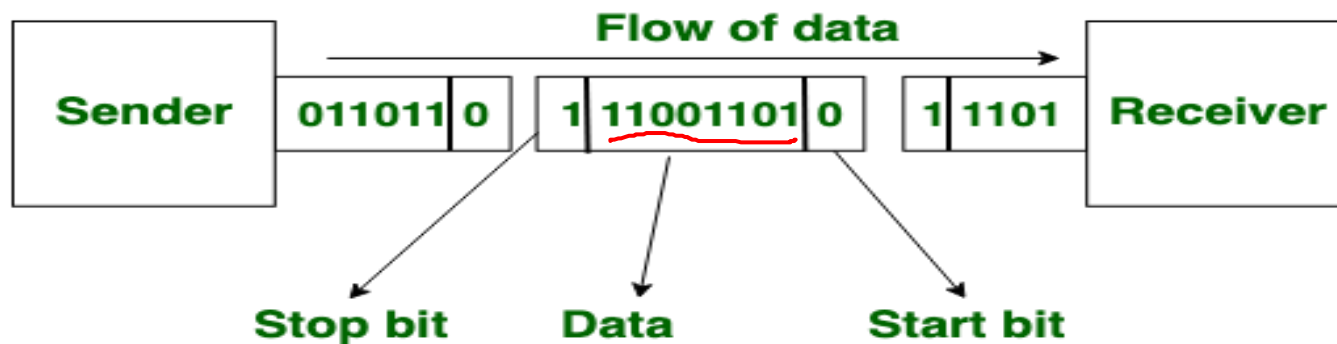
- Data is sent in form of blocks or frames and uses full duplex type and synchronization is compulsory between sender and receiver.
- There is no gap present between data.
- It is more efficient, fast and more reliable than asynchronous transmission to transfer the large amount of data.
- Synchronized clocks for the information of new bytes



Synchronous Transmission

Asynchronous Transmission [9]

- Data is sent in form of byte or character and this transmission is the half duplex
- Start bits(0) and stop bits(1) are added with data. It does not require synchronization.
- No need of synchronized clocks and slow transmission
- Time interval of transmission is random

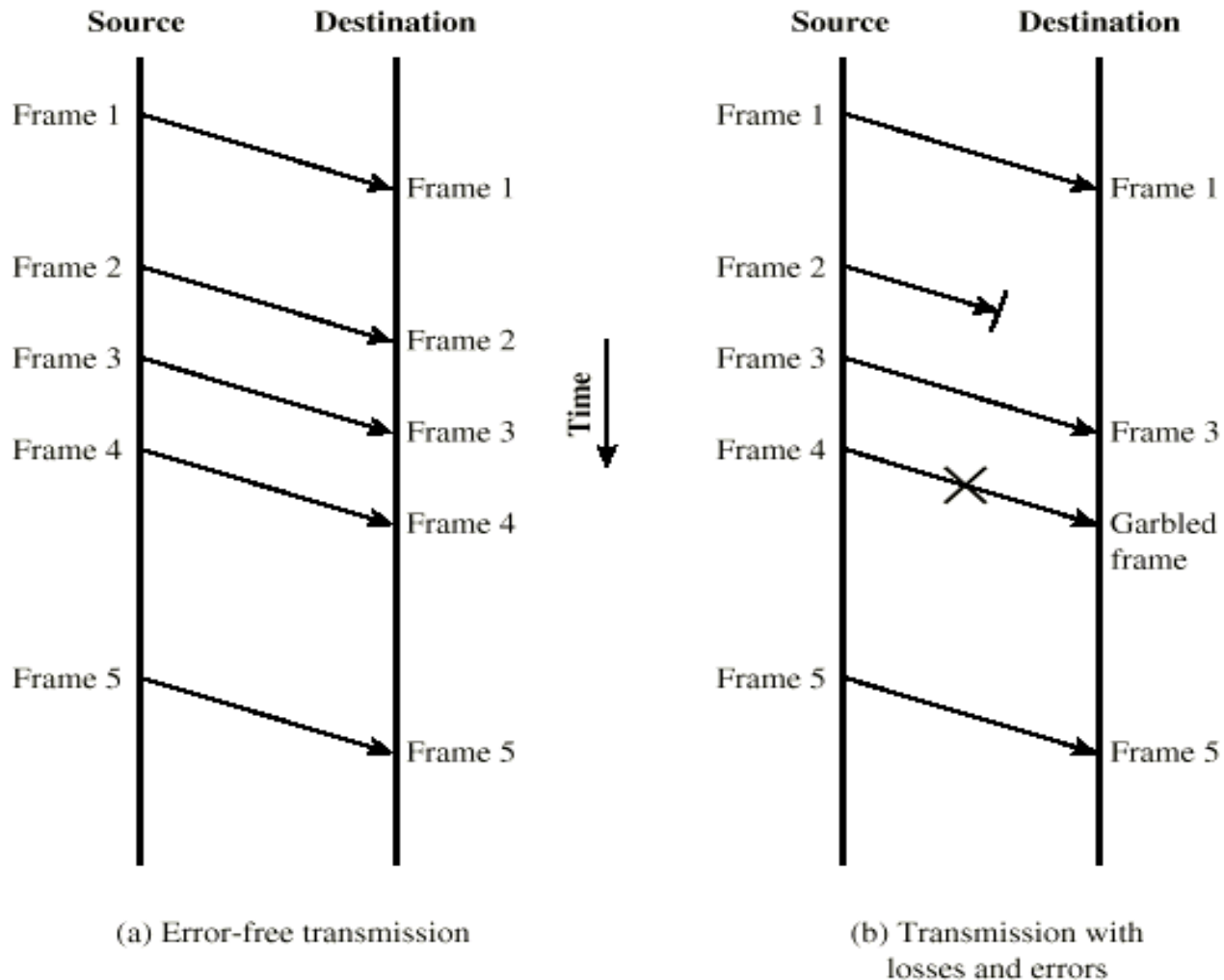


Asynchronous Transmission

Flow Control

- Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment
- Ensuring the sending entity does not overwhelm the receiving entity
 - Preventing buffer overflow
- Transmission time
 - Time taken to emit all bits into medium
- Propagation time
 - Time for a bit to traverse the link

Model of Frame Transmission [1]



Stop and Wait

- Source transmits frame
- Destination receives frame and replies with acknowledgement
- Source waits for ACK before sending next frame
- Destination can stop flow by not sending ACK
- Works well for a few large frames

Contd.

- Large block of data may be split into small frames
 - Limited buffer size
 - Errors detected sooner (when whole frame received)
 - On error, retransmission of smaller frames is needed
 - Prevents one station occupying medium for long periods
- Stop and wait becomes inadequate

Contd.

- R: data rate of the link , in bps
- d= length , distance of the link in meters
- V= propagation speed in m/s
- B: length of the link in bits

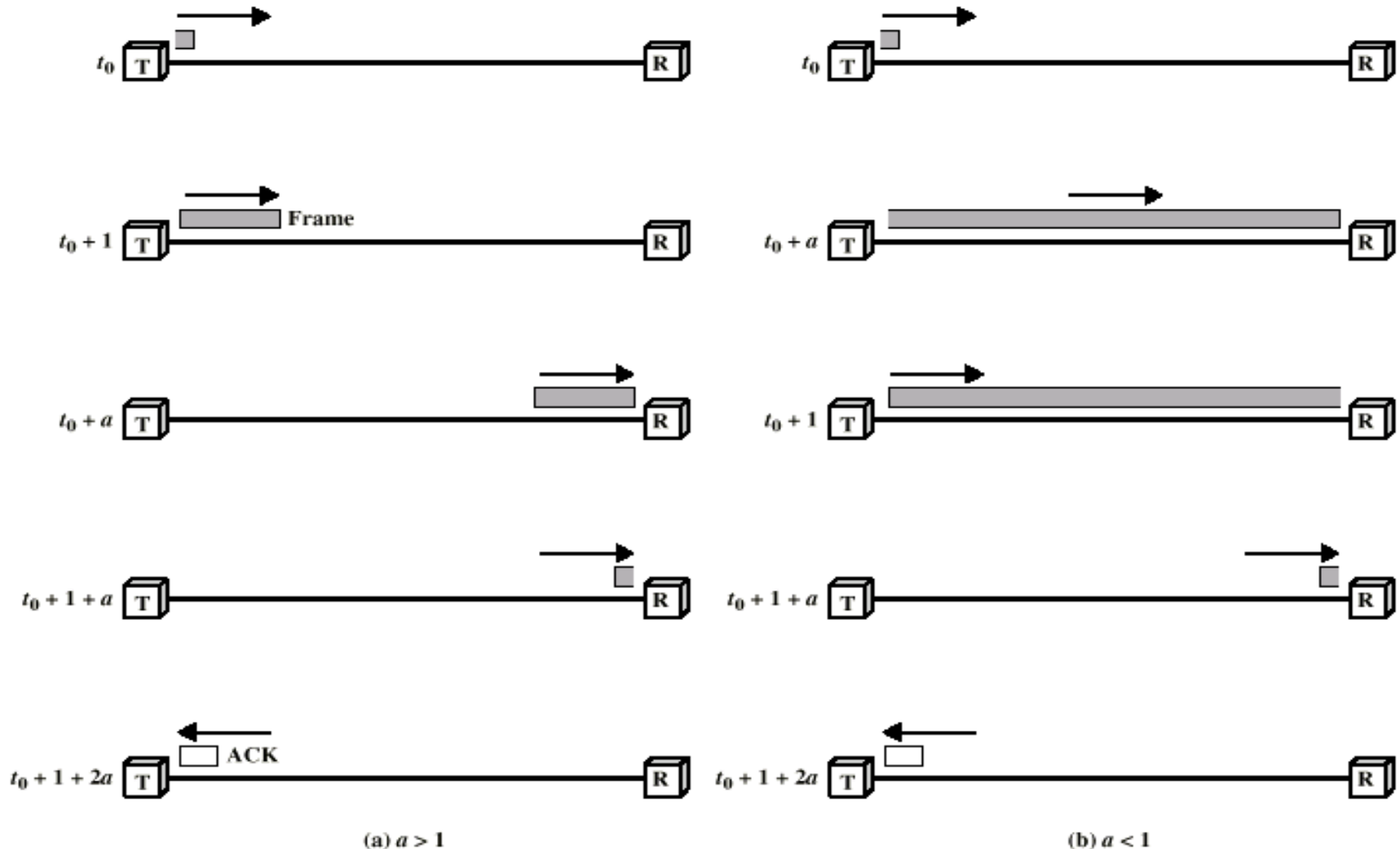
$$B = R * d / V$$

Handwritten red text: $\frac{\text{bit}}{\text{sec}} \times \text{sec}$

- $a = B / L = \text{propagation delay} / \text{transmission delay}$

where L is no of bits in the frame(frame Length)

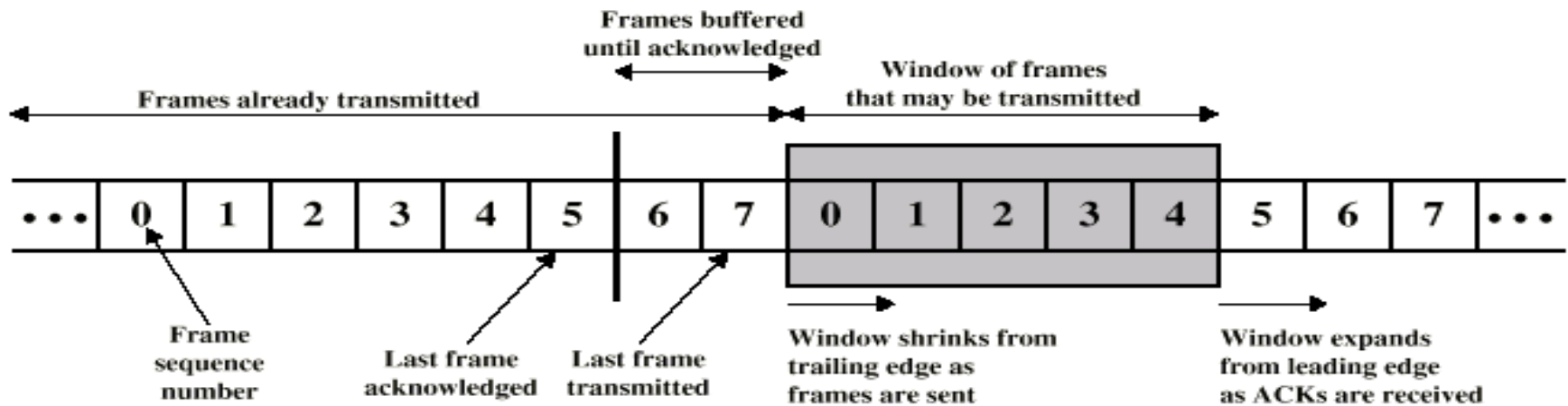
Stop and Wait Link Utilization [1]



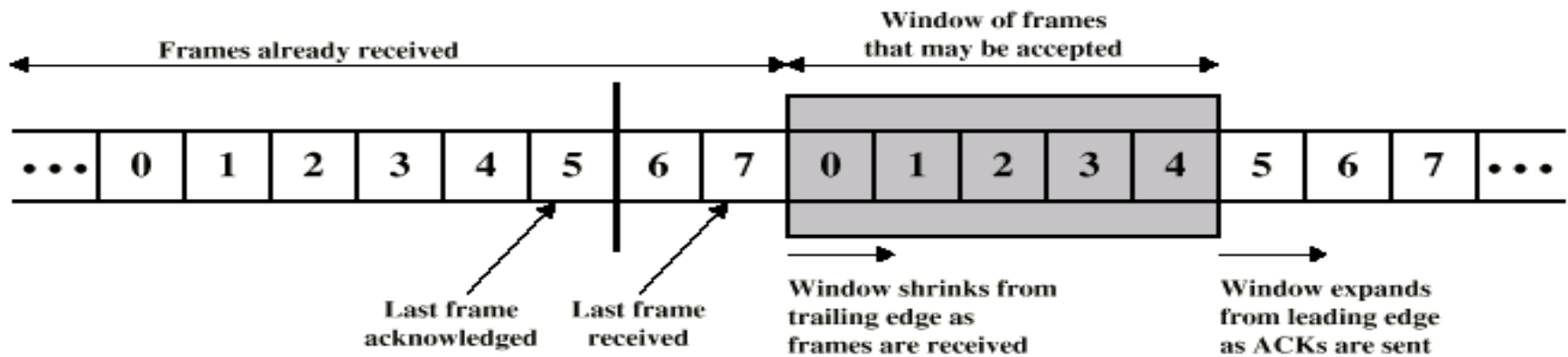
Sliding Windows Flow Control

- Allow multiple frames to be in transit
- Receiver has buffer W long
- Transmitter can send up to W frames without ACK
- Each frame is numbered
- ACK includes number of next frame expected
- Sequence number bounded by size of field (k)
 - Frames are numbered modulo 2^k
 - So it can vary from 0 to $2^k - 1$

Sliding Window Diagram [1]



(a) Sender's perspective



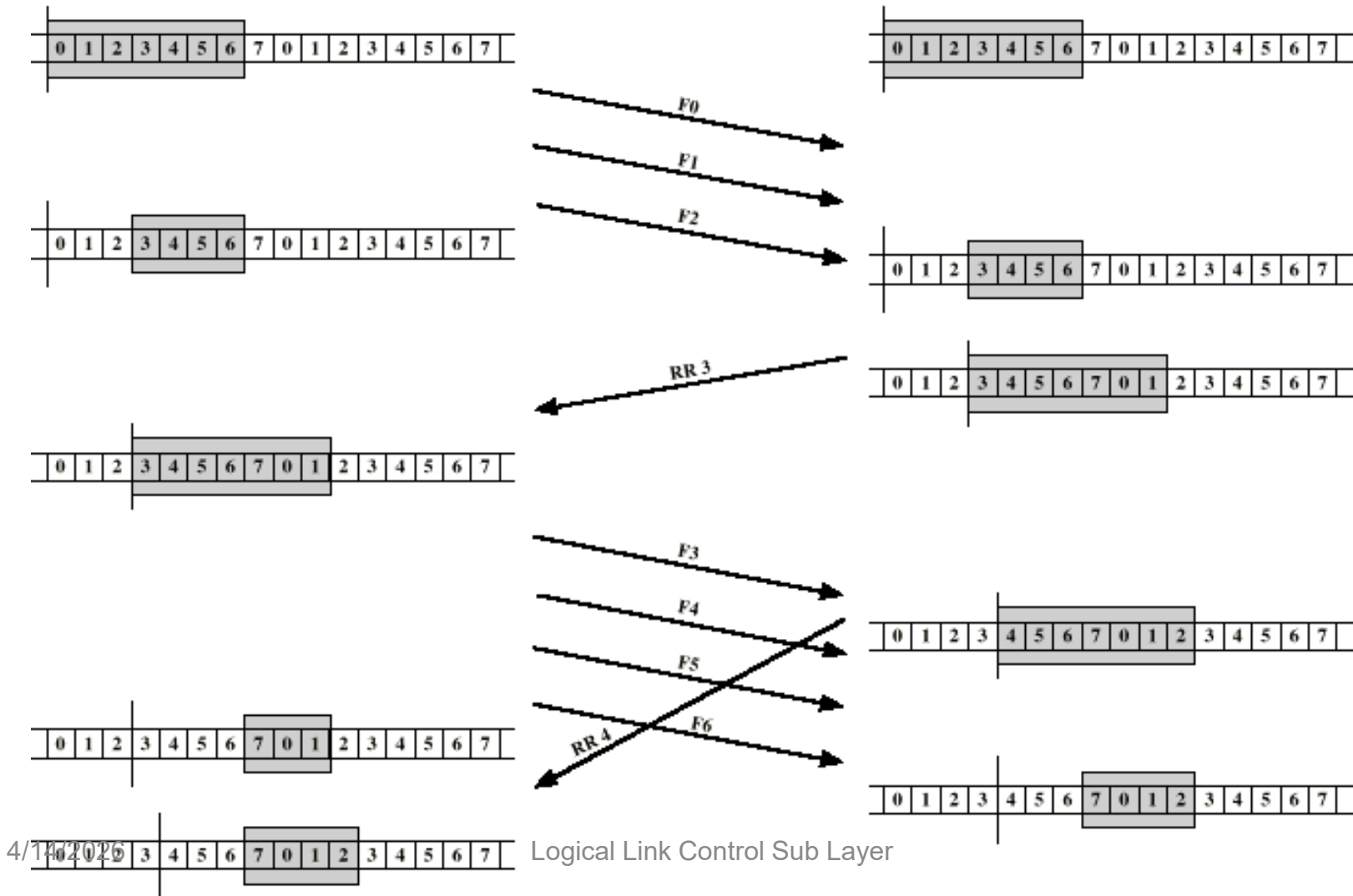
(b) Receiver's perspective

Why window 7?

Example Sliding Window [1]

Source System A

Destination System B



Performance Evaluation

- Stop-Wait Protocol-
 - Maximum utilization of a link, $U = 1/(1+2a)$
 - a is the ratio of propagation delay to transmission delay
- Sliding Window Protocol-
 - Link utilization, $U = W/(1+2a)$, if $W < 2a+1$
 - Link utilization, $U = 1$, if $W \geq 2a+1$
 - W is the window size

Error Control

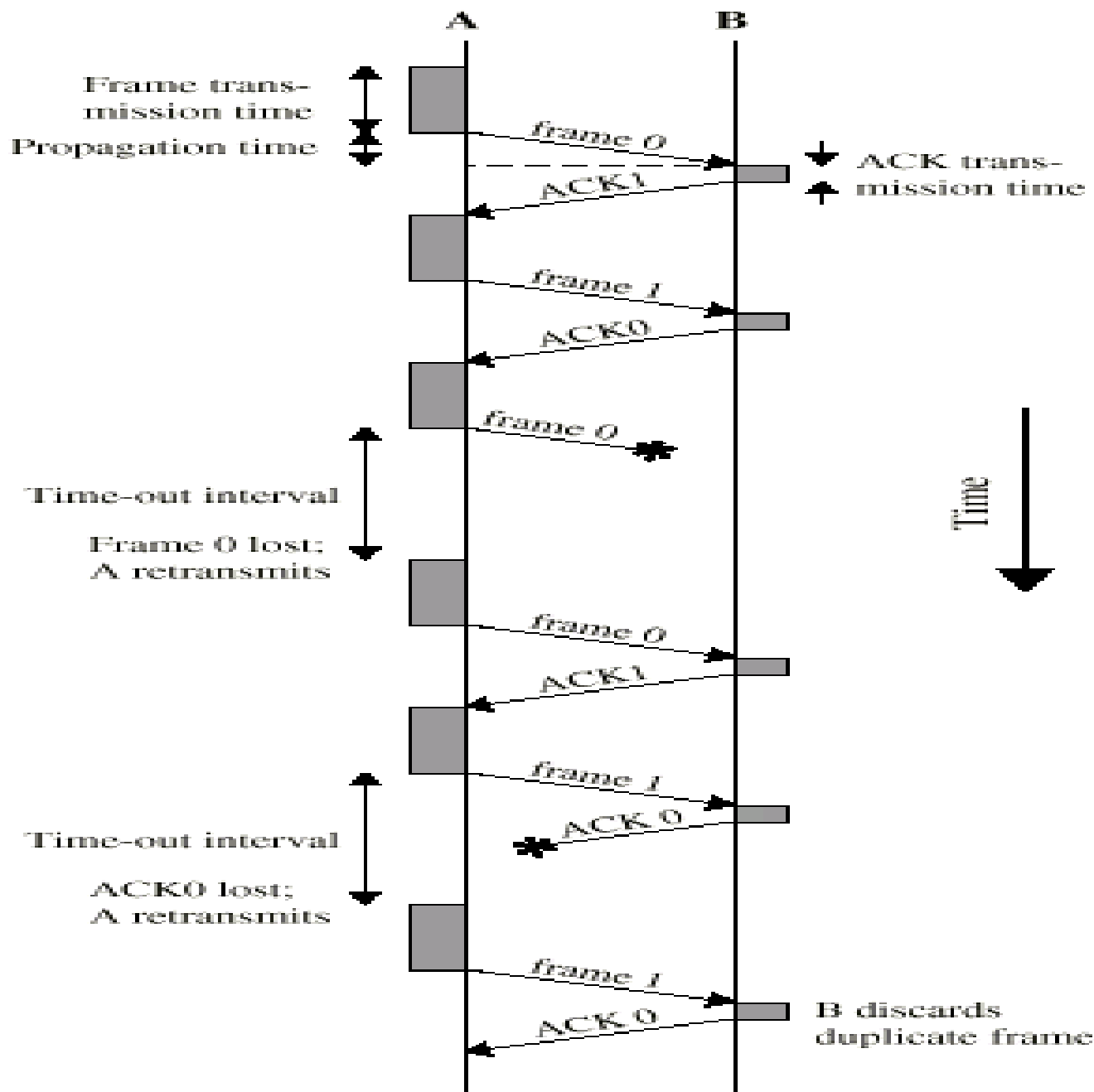
- *Error control in the data link layer is based on automatic repeat request, which is the retransmission of data*
- Error can be caused as:
 - Lost frames
 - Damaged frames
- **Error Control-** It can be controlled by followings
 - Error detection
 - Positive acknowledgment
 - Retransmission after timeout
 - Negative acknowledgement and retransmission

Automatic Repeat Request (ARQ)

- Collectively , these mechanisms are referred as ARQ
- The effect of ARQ is to turn an unreliable data link into a reliable one.
- Three version are:
 - Stop and wait
 - Go back N
 - Selective reject (selective retransmission)

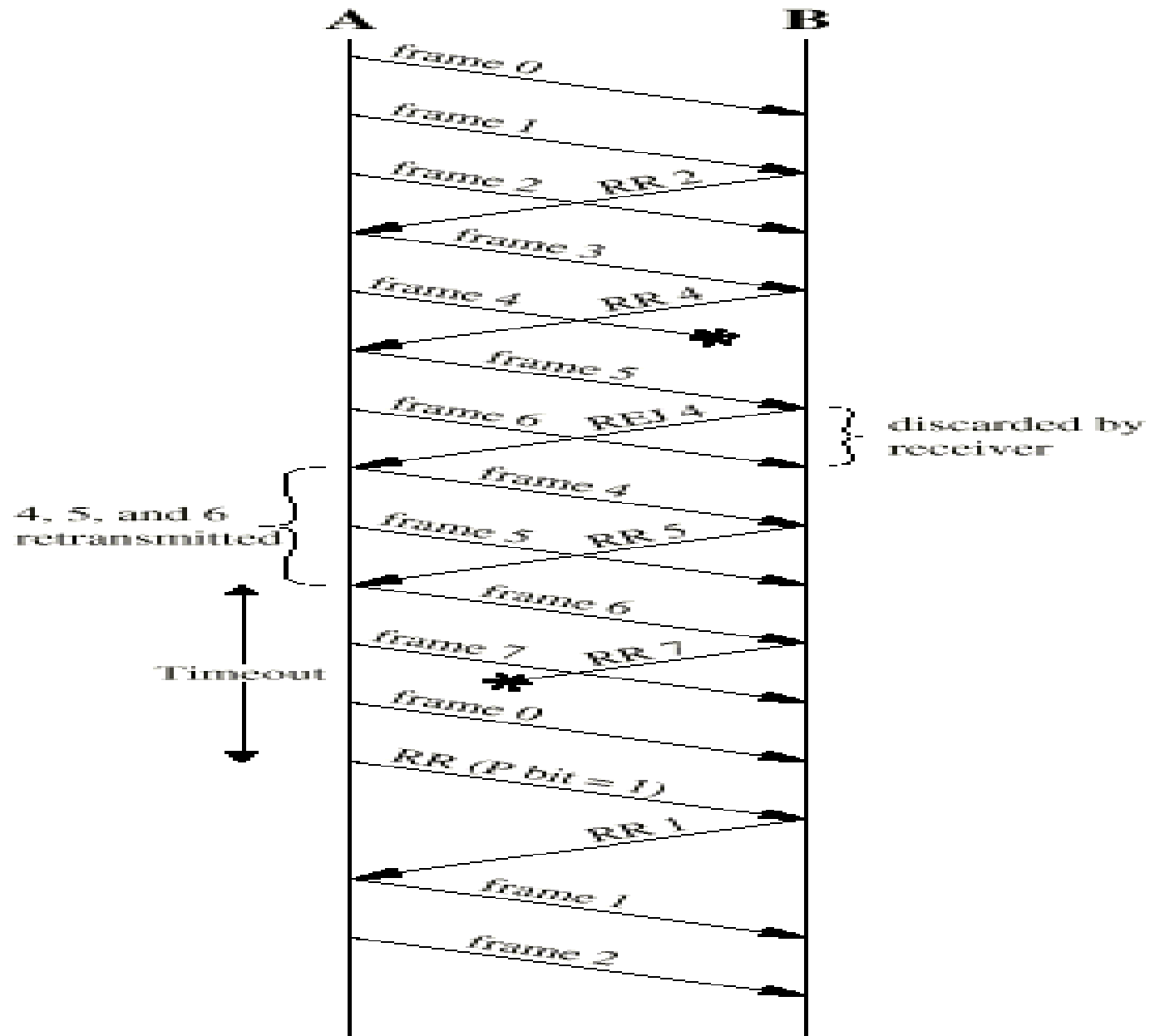
Stop and Wait ARQ [1]

- Source transmits single frame
- Wait for ACK
- If received frame is damaged, discard it
 - Transmitter has timeout
 - If no ACK within timeout, retransmit
- If ACK damaged, transmitter will not recognize it
 - Transmitter will retransmit
 - Receiver gets two copies of frame
 - Use ACK0 and ACK1



Go Back N ARQ [1]

- Based on sliding window
- If no error, ACK is as usual with next frame expected
- Use window to control number of outstanding frames
- If error, reply with rejection
 - Discard that frame and all future frames until error frame received correctly
 - Transmitter must go back and retransmit that frame and all subsequent frames



Go Back N - Damaged Frame

- Receiver detects error in frame i
- Receiver sends rejection- i
- Transmitter gets rejection- i
- Transmitter retransmits frame i and all subsequent frames

Go Back N - Lost Frame(1)

- Frame i lost
- Transmitter sends $i+1$
- Receiver gets frame $i+1$ out of sequence
- Receiver send reject i
- Transmitter goes back to frame i and retransmits

Go Back N - Lost Frame (2)

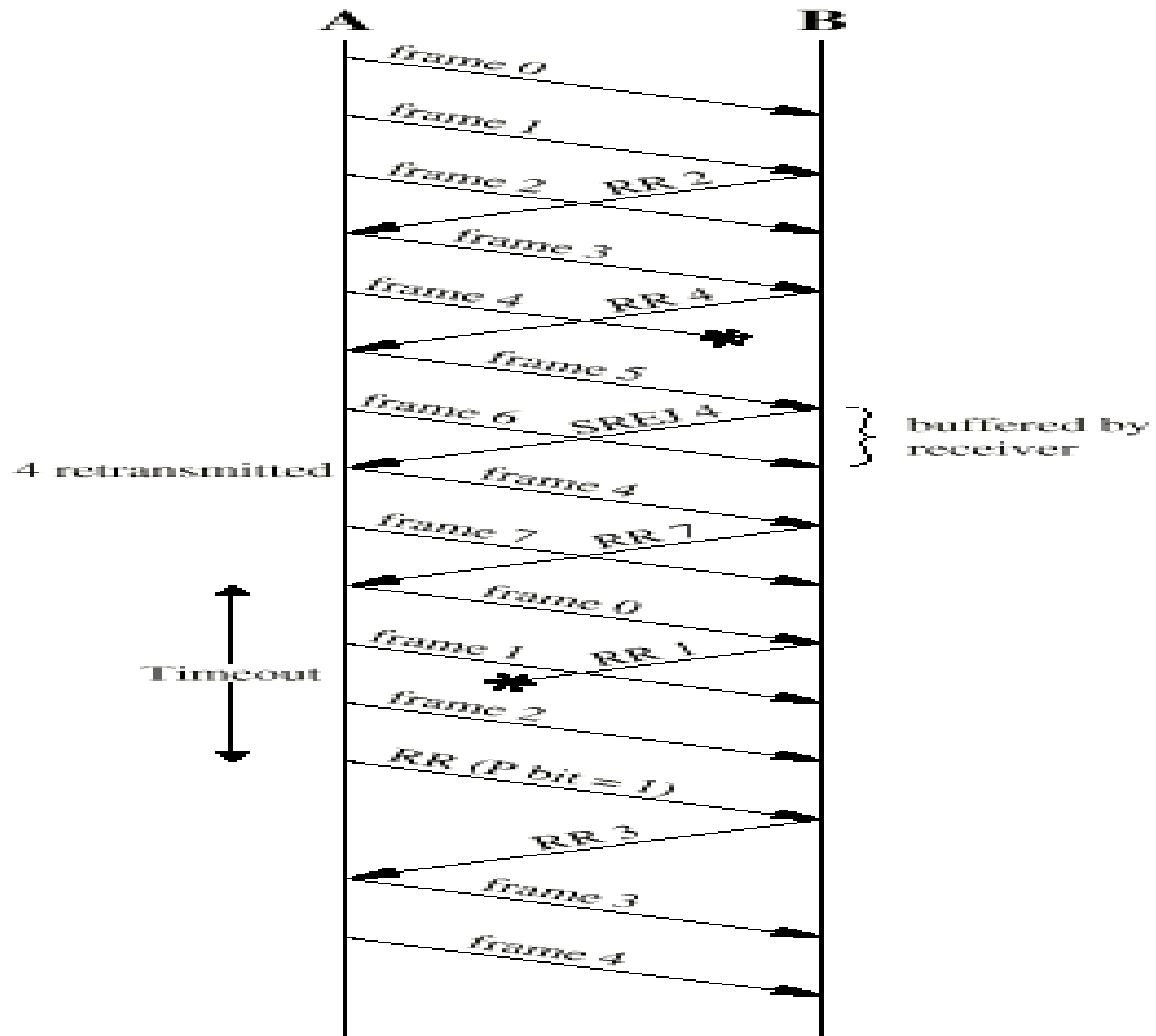
- Frame i lost and no additional frame sent
- Receiver gets nothing and returns neither acknowledgement nor rejection
- Transmitter times out and sends acknowledgement frame with P bit set to 1
- Receiver interprets this as command which it acknowledges with the number of the next frame it expects (frame i)
- Transmitter then retransmits frame i
- *Alternatively, sender could just retransmit frame i when its timer expires.*

Go Back N - Damaged Acknowledgement

- Receiver gets frame i and send acknowledgement $(i+1)$ which is lost
- Acknowledgements are cumulative, so next acknowledgement $(i+n)$ may arrive before transmitter times out on frame i
- If transmitter times out, it sends acknowledgement with P bit set as before
- This can be repeated a number of times before a reset procedure is initiated

Selective Reject [1]

- Also called selective retransmission
- Only rejected frames are retransmitted
- Subsequent frames are accepted by the receiver and buffered
- Minimizes retransmission
- Receiver must maintain large enough buffer
- More complex logic in transmitter



References

1. William Stallings, “Data and Computer Communications”, Seventh Edition, PHI 2004.
2. Andrew S. Tanenbaum, ”Computer Networks” 4th Edition PHI
3. B. A. Fourozan, “TCP/IP Protocol Suite”, 3rd Edition, Singapore, McGrawHill, 2004.
4. L. L. Peterson and B. S. Davie, Computer Networks-A System Approach, Elsevier.
5. B. A. Fourozan, “Data Communications and Networking”, 4th Edition, Singapore, McGrawHill, 2004.
6. James F. Kurose, Keith W. Ross, “Computer Networking: A Top-Down Approach Featuring the Internet”, 3rd Edition , Pearson Education 2009.
7. https://gaia.cs.umass.edu/kurose_ross/ppt.htm
8. PPT available for the respective books
9. <https://www.geeksforgeeks.org/difference-between-synchronous-and-asynchronous-transmission/>

Thank You